

Erasmus Mundus Joint Master Artificial Intelligence
for Sustainable Societies (AISS)

Topics on Machine Learning and its Applications

Model evaluation

1st part

Pedro Strecht



Model evaluation (1st part)

Introduction

Loss functions

The confusion matrix

Performance metrics for classification

Case studies for model evaluation

Model evaluation

Introduction

In CRISP-DM **Model evaluation** corresponds to **Evaluation**

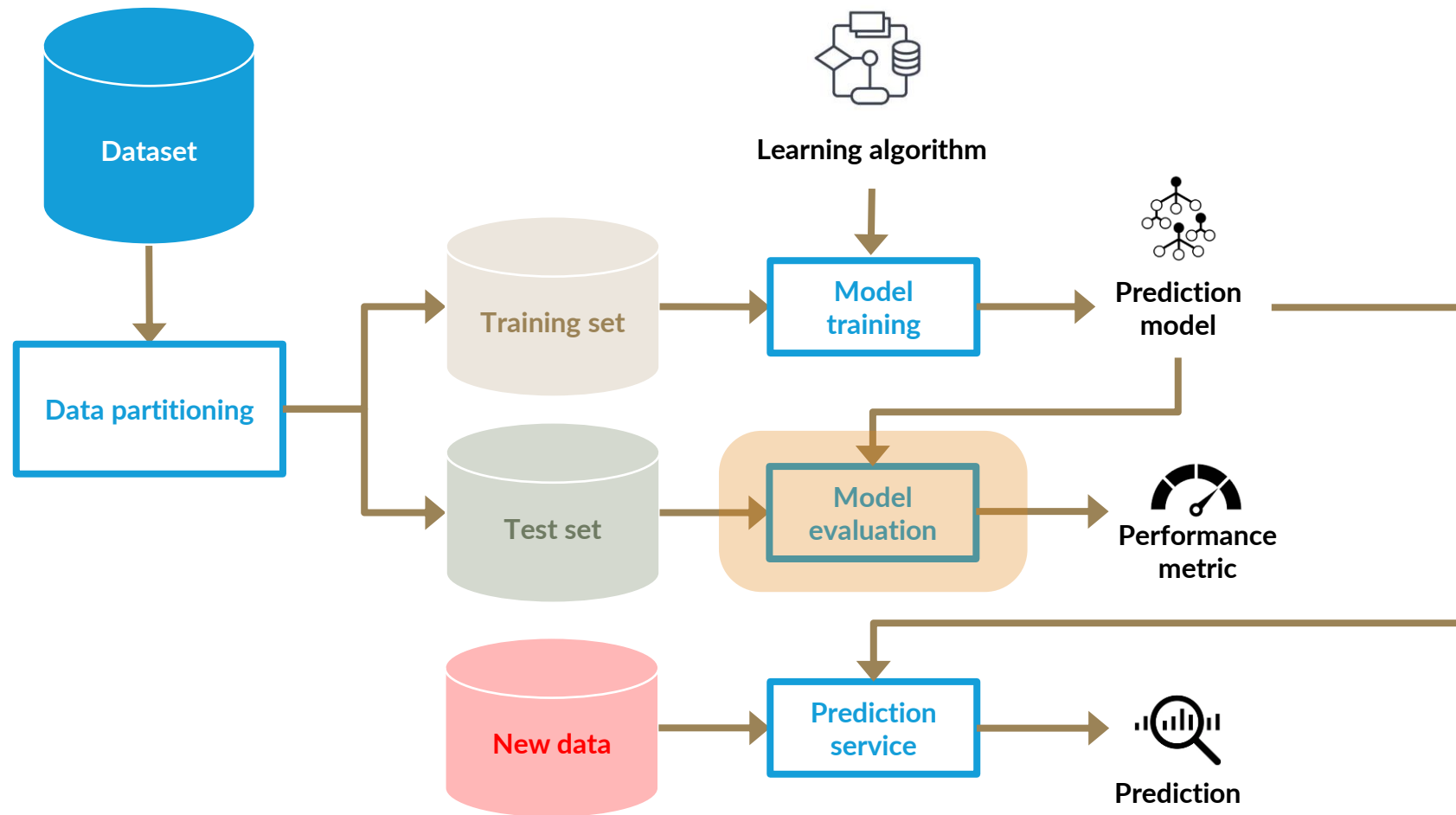
- The **evaluation phase** focuses on assessing the quality and validity of the selected model(s)
- While initial assessments may indicate a high-quality model, this phase delves deeper, **employing various evaluation methods** to determine the model's true performance and identify the best candidate among alternatives
- This involves analyzing the model's predictive accuracy, stability, and generalizability using methods such as hold-out or cross-validation and performance metrics
- Beyond technical evaluation, this phase also involves a critical review of the model's alignment with business objectives



Model evaluation

Introduction

General experimental set-up



Model evaluation

Introduction

■ What is model evaluation?

- Typically, it is the process of assessing a machine learning model's performance on new, unseen data
- Helps determine the model's accuracy, reliability, and ability to generalize to real-world scenarios
- It can encompass other criteria beyond predictive performance

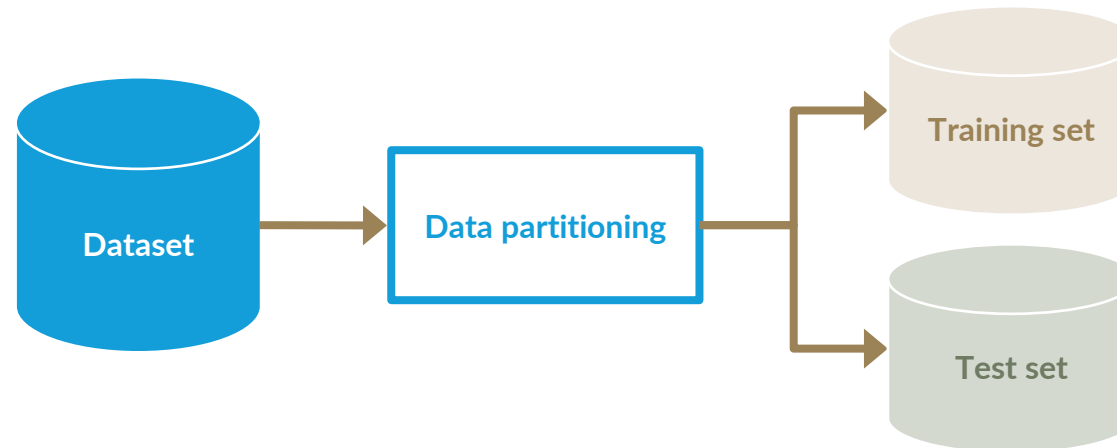
■ Why is model evaluation important?

- Identifies model strengths and weaknesses
- Prevents overfitting, where the model performs well on training data but poorly on new data
- Guides model selection and hyperparameter tuning
- Builds trust and confidence in the model's predictions

Model evaluation

Introduction

- **Datasets** form the foundation for developing and evaluating predictive models
- Typically, a dataset is partitioned into two subsets:
 - a **training set**, where the model learns from the data
 - a **test set**, where the model's performance is evaluated



Model evaluation

Introduction

- A **test set**, denoted as TS , is a dataset used to evaluate a prediction model, and similar, in structure to the training set used to train it
- The set of **independent variables**, in a test set must be the same as the ones used for the training set

$$X_{TS} = X_{Tr} = \{x_1, x_2, \dots, x_d\}$$

- A critical assumption of model evaluation is that the test set must consist **exclusively** of examples that were **not used** during the model's training
 - This condition ensures an unbiased assessment of the model's ability to generalize to new, previously **unseen data**

Model evaluation

Introduction

- An example within a test set includes values for both **independent variables** ($v_{i,j}$) and the **target variable** (y_i)

$$e_i = (X_{Ts}, y_i) = (v_{i,1}, v_{i,2}, \dots, v_{i,d}, y_i)$$

- The function that uses a prediction model M to predict the target variable for a given example e is commonly referred to as **Predict**

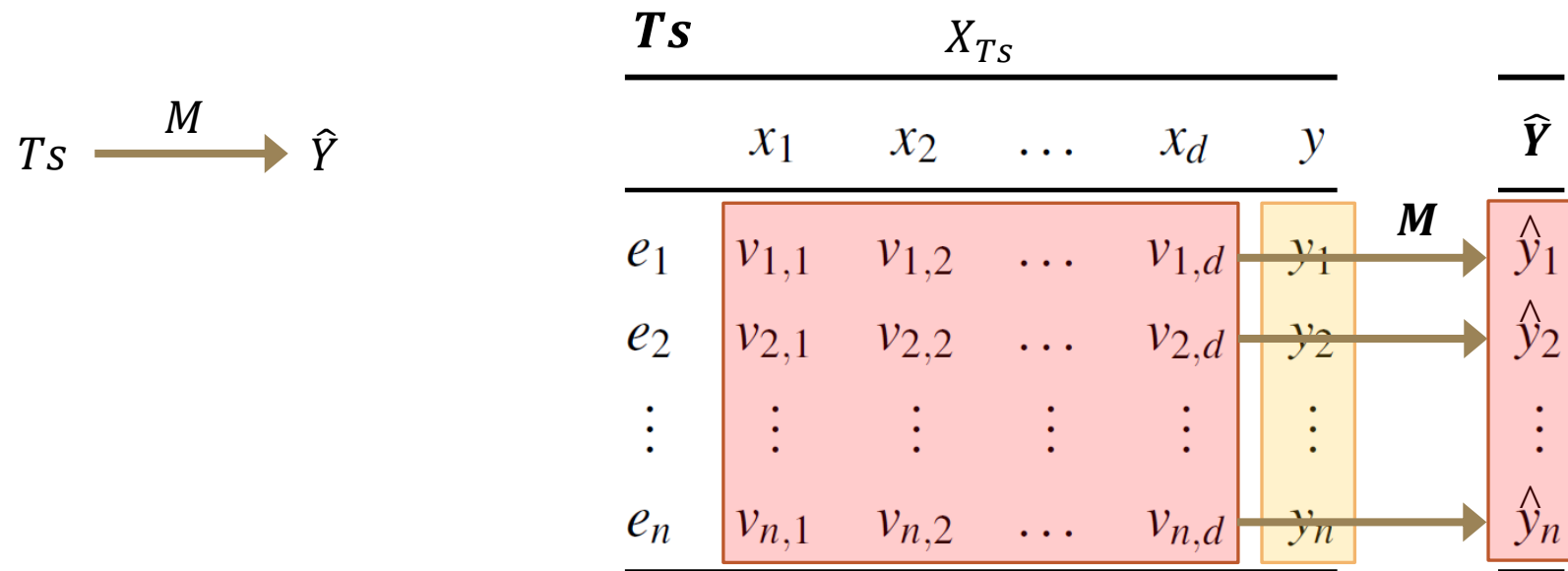
$$\hat{y} = \text{Predict}(M, e)$$

- The result of this operation is a **prediction**, denoted as $\hat{y}$
- All the predictions made on a test set are held in an **array of predictions**, denoted as $\hat{Y}$

Model evaluation

Introduction

- A test set can be structured in a tabular format, with columns representing the variables and rows representing individual examples
- During evaluation, the model generates predictions for each example in the test set



- n denotes the size of the test set ($n = |Ts|$), d denotes the dimension of the test set ($d = |X_{Ts}|$)

Model evaluation

Loss functions

- The goal is to minimize a **loss function** (or *error function*) specific to the type of target variable being predicted, which measures the **errors** made by the model in its predictions
- A **loss function** quantifies the discrepancy or error in the target variable for a given example between:
 - a model's predicted value ($\hat{y}$)
 - the actual true value (y)
- The nature of the target variable determines the choice of an appropriate contrast function, denoted by γ , which calculates the error
- By analyzing these discrepancies, metrics are derived to provide a quantitative assessment of the model's performance

Model evaluation

Loss functions

- For **classification problems**, the loss function takes the form of an **indicator function**
 - This function assigns a value of 1 if a specific proposition (p) is true and 0 otherwise, translating logical comparisons into numerical values

$$I[p] = \begin{cases} 1 & \text{if } p \text{ is true} \\ 0 & \text{if } p \text{ is false} \end{cases}$$

- In the context of evaluation for a given example:

$$\gamma = (M, (X, y)) = I[y \neq \hat{y}]$$

- The error is 1 if the model's prediction ($\hat{y}$) differs from the actual target variable value (y)
- The error is 0 if the prediction is correct

Model evaluation

Loss functions

- For regression problems, the errors are continuous values
 - The error is the **difference** between the model's prediction ($\hat{y}$) from the actual target variable value (y)
 - While other heuristics can be considered, the errors are typically squared to ensure that both positive and negative errors are treated equally and that larger errors have a greater impact
- In the context of evaluation for a given example:

$$\gamma = (M, (X, y)) = (y - \hat{y})^2$$

Model evaluation

Loss functions

- One of the goals of a model (whether for classification or regression) is to **minimize the loss function**
- The **empirical average loss of a model**, denoted as $\text{Loss}(M)$, is the average of the result of a loss function across n examples in a test set

$$\text{Loss}(M) = \frac{1}{n} \sum_{i=1}^n \gamma(M, (X_i, y_i))$$

- In summary:
 - For classification problems, this function corresponds to the **error rate**, which is the proportion of incorrect predictions relative to all examples in the test set
 - In regression problems, this function corresponds to the **mean squared error**, which is the sum of all squared errors in the predictions made in the test set

Model evaluation

The confusion matrix

- In binary classification, the **positive class** (denoted as $\oplus$) represents the event, condition, or outcome of primary interest (e.g., fraud, disease, dropout)
- The **negative class** (denoted as $\ominus$) represents the alternative outcome (i.e., the absence of the target event)
- The designation of the positive class is not inherent to the data; it is a modelling choice driven by the **problem context and decision objectives**
- The choice of positive class should reflect the practical cost of errors

Model evaluation

The confusion matrix

- Example:
 - A model that predicts whether a student will fail a course
 - The **target variable** is y = “approval”, with two possible classes: “passed” or “failed”
 - If the primary goal is to evaluate the model’s ability to identify students at risk of failing:
 - $\oplus$ = “failed”
 - $\ominus$ = “passed”
 - The **positive class** is the class capturing the phenomena of interest
 - Any other can be regarded as **negative class**

Model evaluation

The confusion matrix

Classification problem	$\oplus$	$\ominus$
Credit card fraud detection	Fraudulent transaction	Legitimate transaction
Medical diagnosis (cancer screening)	Patient has cancer	Patient does not have cancer
Student dropout prediction	Student drops out	Student completes studies
Spam detection	Email is spam	Email is not spam
Equipment failure prediction	Machine fails	Machine operates normally
Customer churn prediction	Customer churns	Customer stays
Disease testing (e.g., COVID-19)	Test is positive	Test is negative
Detect fraud transaction	Transaction is malicious	Transaction is normal
Manufacturing quality control	Defective product	Non-defective product

Model evaluation

The confusion matrix

- From these concepts, the **examples** within a test set

$$Ts = \{e_1, \dots, e_n\}$$

$$e_i = (X_{Ts}, y_i) = (v_{i,1}, v_{i,2}, \dots, v_{i,d}, y_i)$$

$$\hat{y}_i = \text{Predict}(M, e_i)$$

can be categorized based on:

- Their **actual** and **predicted** target variable values
- They belong to the **positive** or the **negative** class

Ts	X_{Ts}					
	x_1	x_2	$\dots$	x_d	y	$\hat{Y}$
e_1	$v_{1,1}$	$v_{1,2}$	$\dots$	$v_{1,d}$	y_1	$\hat{y}_1$
e_2	$v_{2,1}$	$v_{2,2}$	$\dots$	$v_{2,d}$	y_2	$\hat{y}_2$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
e_n	$v_{n,1}$	$v_{n,2}$	$\dots$	$v_{n,d}$	y_n	$\hat{y}_n$

Model evaluation

The confusion matrix

- This categorization allows for the definition of the following metrics:
 - **Actual positives** (P_a) – The number of examples in a test set where the **actual value** of the target variable is the **positive class**

$$P_a = |\{e_i: e_i \in Ts \wedge y_i = \oplus\}|$$

- **Actual negatives** (N_a) – The number of examples in a test set where the **actual value** of the target variable is not the **positive class**

$$N_a = |\{e_i: e_i \in Ts \wedge y_i \neq \oplus\}|$$

Model evaluation

The confusion matrix

- This categorization allows for the definition of the following metrics:
 - **Predicted positives** (P_p) – The number of examples in a test set where the **predicted value** of the target variable by the model is the **positive class**

$$P_p = |\{e_i: e_i \in Ts \wedge \hat{y}_i = \oplus\}|$$

- **Predicted negatives** (N_p) – The number of examples in a test set where the **predicted value** of the target variable by the model is not the **positive class**

$$N_p = |\{e_i: e_i \in Ts \wedge \hat{y}_i \neq \oplus\}|$$

Model evaluation

The confusion matrix

- In model evaluation, it is essential to differentiate between **successful predictions** and **errors** made by the model
- Building on these concepts, the examples in the test set can be further categorized based on whether the **actual** and **predicted** target variable values **match** or **differ**, specifically regarding the prediction of the positive class:
 - **True positives**
 - **True negatives**
 - **False positives**
 - **False negatives**

Model evaluation

The confusion matrix

- This distinction is based on the **agreement** between the model's predictions and the true values of the target variable
 - **True positives** and **True negatives** represent successful predictions, meaning the prediction is **aligned with reality**
 - **False negatives** and **False positives** represent the errors made by the model, meaning the prediction is **different than reality**

Model evaluation

The confusion matrix

- Cases when the model makes correct model predictions:
 - **True positives** (TP) – The number of examples in a test set **correctly** predicted by the model as **belonging to the positive class**

$$TP = |\{e_i: e_i \in Ts \wedge y_i = \oplus \wedge y_i = \hat{y}_i\}|$$

- **True negatives** (TN) – The number of examples in a test set **correctly** predicted by the model as **not belonging to the positive class**

$$TN = |\{e_i: e_i \in Ts \wedge y_i \neq \oplus \wedge y_i = \hat{y}_i\}|$$

Model evaluation

The confusion matrix

- Cases when the models makes mistakes:
 - **False negatives** (FN) – The number of examples in a test set **incorrectly** predicted by the model as **not belonging to the positive class** (as they, in fact, belong to the positive class)

$$FN = |\{e_i: e_i \in Ts \wedge y_i = \oplus \wedge y_i \neq \hat{y}_i\}|$$

- **False positives** (FP) – The number of examples in a test set **incorrectly** predicted by the model as **belonging to the positive class** (as they, in fact, do not belong to the positive class)

$$FP = |\{e_i: e_i \in Ts \wedge y_i \neq \oplus \wedge y_i \neq \hat{y}_i\}|$$

Model evaluation

The confusion matrix

- These class-specific counts are often presented in a contingency table known as a **confusion matrix**

		Predicted values		Total
		Positive	Negative	
Actual values	Positive	<i>TP</i>	<i>FN</i>	P_a
	Negative	<i>FP</i>	<i>TN</i>	N_a
Total		P_p	N_p	n

- This matrix summarizes the model's performance by capturing the **agreements** (TP and TN) and **disagreements** (FN and FP) between its predictions and the actual values

Model evaluation

The confusion matrix

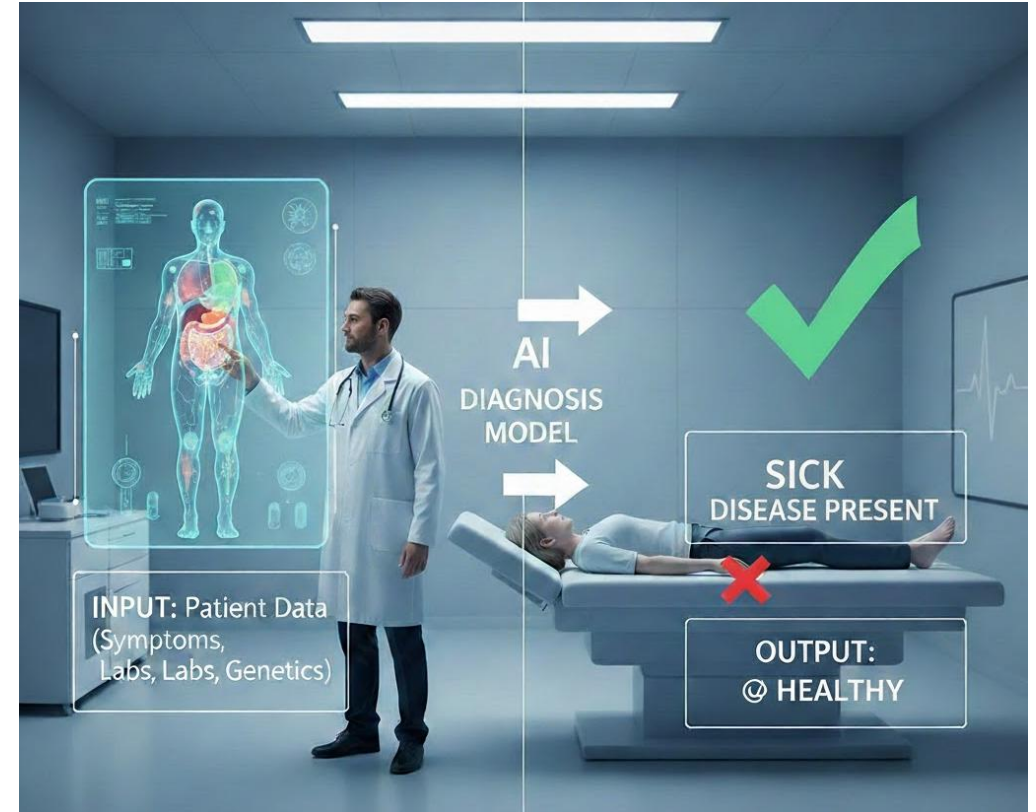
		Predicted values		Total
		Positive	Negative	
Actual values	Positive	<i>TP</i>	<i>FN</i>	P_a
	Negative	<i>FP</i>	<i>TN</i>	N_a
Total		P_p	N_p	n

- In the case of **False positives** (*FP*), the model raises a **false alarm**, incorrectly classifying an example belonging to the negative class as positive
 - Often referred to as a **Type I error**
- In the case of **False negatives** (*FN*), the model **misses** an example belonging to the positive class, incorrectly classifying it as negative
 - Often referred to as a **Type II error**

Model evaluation

The confusion matrix

- **Medical diagnosis example**
 - Predict whether a patient has a disease or not
- Good decisions:
 - **True positive:** classify a sick patient as **Sick**
 - **True negative:** classify a healthy patient as **Healthy**
- Errors:
 - **False positive:** classify a healthy patient as **Sick**
 - **False negative:** classify a sick patient as **Healthy**



Model evaluation

The confusion matrix

- Evaluation on a test set

Patient ID	True Label	Predicted Label	Evaluation
1	Sick	Sick	True Positive
2	Sick	Healthy	False Negative
3	Healthy	Sick	False Positive
4	Healthy	Healthy	True Negative
5	Sick	Sick	True Positive
6	Healthy	Healthy	True Negative
7	Sick	Healthy	False Negative
8	Healthy	Sick	False Positive
9	Healthy	Healthy	True Negative
10	Sick	Sick	True Positive

Model evaluation

The confusion matrix

- Confusion matrix from the test set

		Predicted values		Total
		Positive	Negative	
Actual values	Positive	3	2	5
	Negative	2	3	5
Total		5	5	10

- In this case, a **False negative** is more costly than a **False positive**, as sick patients may not receive the necessary treatment

Model evaluation

The confusion matrix

- **Spam e-mail example**

- Predict whether an e-mail is spam or not

- **Good decisions:**

- **True positive:** classify a spam email as **Spam**
- **True negative:** classify a good email as **Good**

- **Errors:**

- **False positive:** classify a good email as **Spam**
- **False negative:** classify a spam email as **Good**



Model evaluation

The confusion matrix

- Evaluation on a test set

Email ID	True Label	Predicted Label	Evaluation
1	Spam	Spam	True Positive
2	Spam	Not Spam	False Negative
3	Not Spam	Spam	False Positive
4	Not Spam	Not Spam	True Negative
5	Spam	Spam	True Positive
6	Not Spam	Spam	False Positive
7	Spam	Not Spam	False Negative
8	Not Spam	Spam	False Positive
9	Not Spam	Not Spam	True Negative
10	Spam	Spam	True Positive

Model evaluation

The confusion matrix

- Confusion matrix from the test set

		Predicted values		Total
		Positive	Negative	
Actual values	Positive	3	2	5
	Negative	3	2	5
Total		6	5	10

- In this case, a **False positive** is more costly than a **False negative**, as important emails may be mistakenly sent to the spam folder

Model evaluation

Performance metrics for classification

- The **confusion matrix** summarizes prediction outcomes (TP , TN , FP , FN)
- There are many metrics that can be directly derived from the confusion matrix
- Usually, the predictive performance of a model is presented using **performance metrics**
- The **overall evaluation of the performance** of a model (with classifier or regressor) relies on a set of performance metrics, differing in criteria as interpretability and robustness, among others
 - Of the metrics, not the models themselves!

		Predicted values		Total
		Positive	Negative	
Actual values	Positive	TP	FN	P_a
	Negative	FP	TN	N_a
Total		P_p	N_p	n

Model evaluation

Performance metrics for classification

- **Accuracy** is the proportion of examples within a test set that the model correctly classifies

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

		Predicted values		Total
		Positive	Negative	
Actual values	Positive	TP	FN	P_a
	Negative	FP	TN	N_a
Total		P_p	N_p	n

- **Error rate** is the complement of accuracy and focuses on the fraction of examples that the model misclassifies

$$ErrorRate = \frac{FP + FN}{TP + TN + FP + FN}$$

		Predicted values		Total
		Positive	Negative	
Actual values	Positive	TP	FN	P_a
	Negative	FP	TN	N_a
Total		P_p	N_p	n

Model evaluation

Performance metrics for classification

- While **Accuracy** and **Error rate** are useful metrics for model evaluation, they have limitations in comprehensively assessing performance
 - Both measure the proportion of correctly/incorrectly classified examples, providing a general overview
- However, **they do not differentiate between types of errors**, i.e., whether they are **False positives** or **False negatives**

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

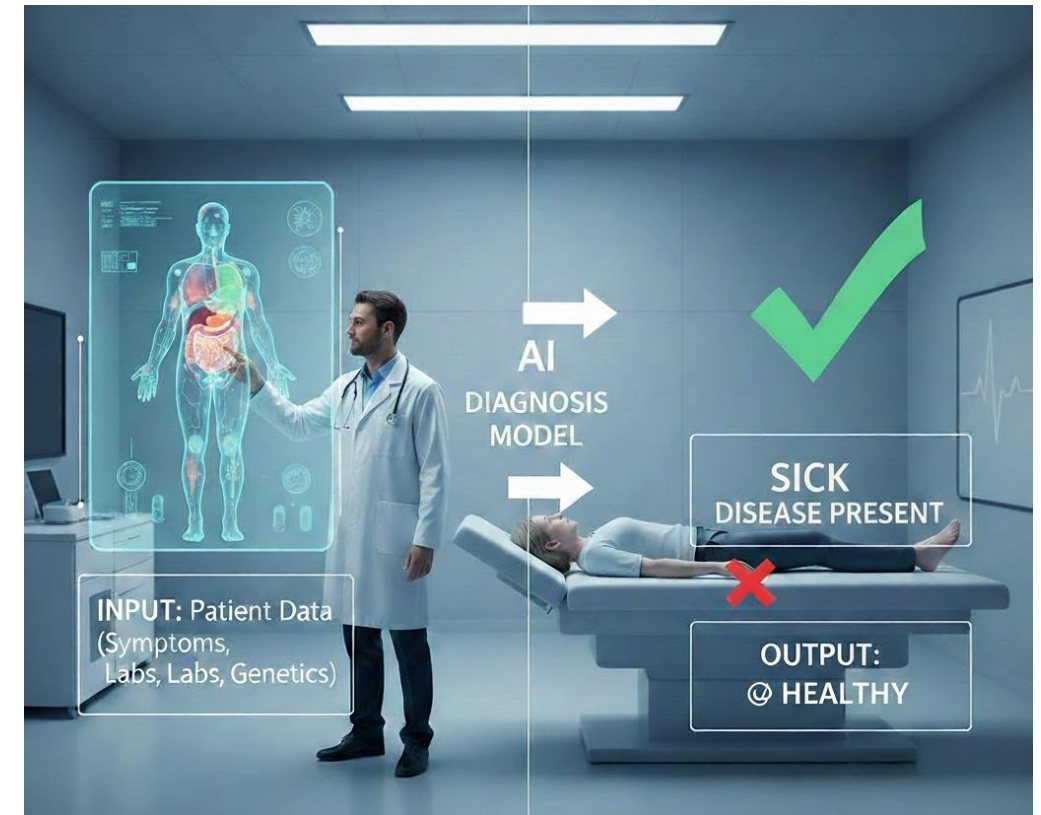
$$ErrorRate = \frac{FP + FN}{TP + TN + FP + FN}$$

- This limitation can be particularly problematic in scenarios where certain errors carry significantly greater costs or consequences compared to others

Model evaluation

Performance metrics for classification

- In domains like **medical diagnosis**, **False negatives** (missing a disease) can have far-reaching negative impacts on patient health and well-being
 - models can be tuned to **reduce the likelihood of false negatives**
 - even if it means sacrificing some true positives (accurately diagnosed patients)
- This might involve setting a more demanding threshold for classifying individuals as having a specific disease



Model evaluation

Performance metrics for classification

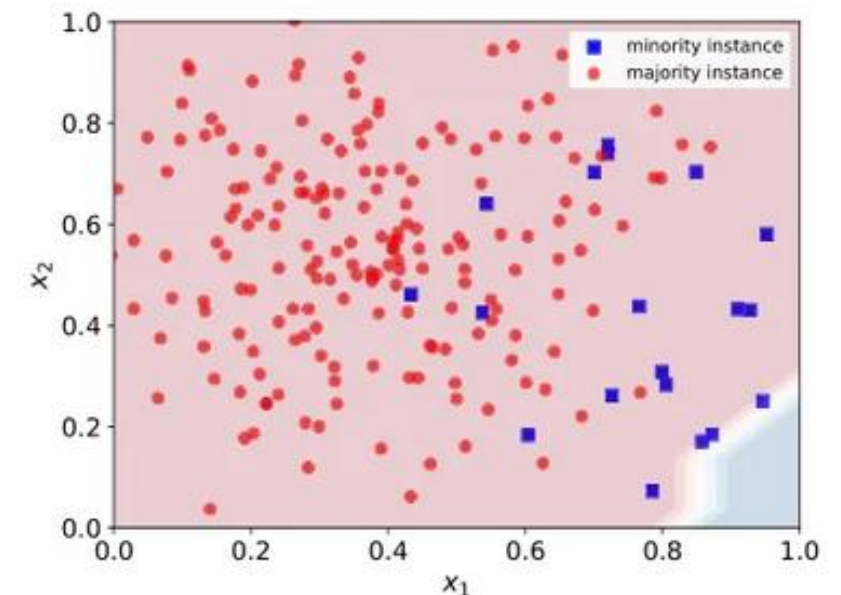
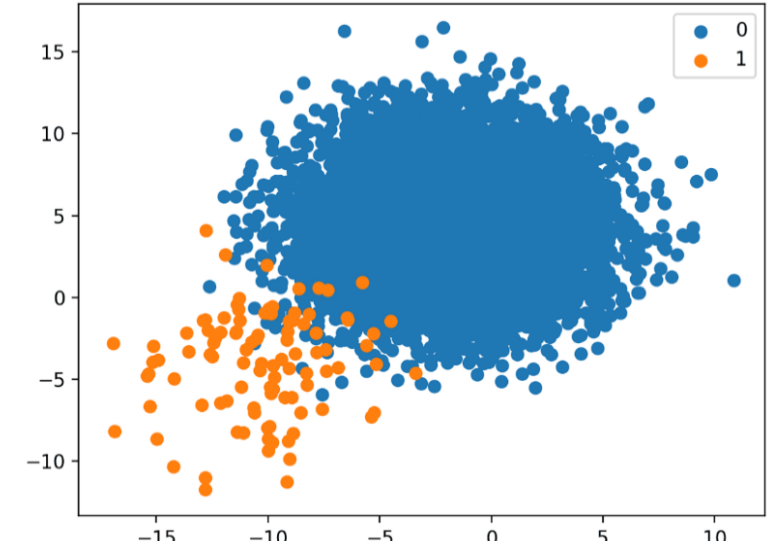
- Conversely, in contexts like **fraud detection**, **False positives** (misidentifying legitimate transactions as fraudulent) can lead to unnecessary investigations and potentially damage the reputation of trusting clients
 - models in such scenarios might be optimized to **minimize false positives**
 - even if resulting in a few missed fraudulent transactions (true negatives)
- This could involve setting a less restrictive threshold for classifying transactions as suspicious



Model evaluation

Performance metrics for classification

- Another critical factor that motivates the distinction between the evaluation of **False positives** and **False negatives** is the **class imbalance problem**
- Predictive models are frequently employed to analyze phenomena where **the positive class is rare** and more challenging to predict compared to examples from other classes
- The dataset is typically dominated by the negative class, with the positive class being referred to as the **minority class**



Model evaluation

Performance metrics for classification

- Example:

- A classification problem to detect spam e-mails uses a classifier that predicts all e-mails as **Good**
- Test set has 90% of good e-mails (majority class, negative class) and 10% of spam (minority class, positive class)

- Confusion matrix:

		Predicted values		Total
		Positive	Negative	
Actual values	Positive	0	10	10
	Negative	0	90	90
Total		0	100	100

- $Accuracy = \frac{TP+TN}{TP+TN+FP+FN} = \frac{0+90}{0+90+0+10} = 0.9$
- Is this a good model?

It is **not** a good model because it performs poorly on the positive class ($TP=0$)!

Model evaluation

Performance metrics for classification

- The inherent bias towards the majority class in imbalanced datasets renders both **Accuracy** and **Error rate** as misleading metrics
- This is because a model can achieve a high accuracy (and low error rate) by simply predicting all examples as the majority class, regardless of its ability to identify and classify minority class examples

$$\text{Accuracy} = \frac{\cancel{TP} + TN}{\cancel{TP} + TN + \cancel{FP} + FN} = \frac{90}{90 + \textcircled{10}} = 0.9$$

The only source of errors is the rare positive class!

Model evaluation

Performance metrics for classification

- Accuracy becomes an **inflated measure** reflecting the model's performance on the dominant class while concealing its shortcomings in handling the minority class
- Consequently, such models might struggle with accurate minority class identification and classification
- To address this problem, there have been developed metrics specifically designed to be **sensitive to the minority class**, considering either actual or predicted values

Model evaluation

Performance metrics for classification

- **Precision**, also known as **Positive Predictive Value (PPV)**, is the proportion of correctly classified positive examples among all predictions made by the model as positive

$$\text{Precision} = \text{PPV} = \frac{TP}{P_p} = \frac{TP}{TP + FP}$$

		Predicted values		Total
		Positive	Negative	
Actual values	Positive	TP	FN	P_a
	Negative	FP	TN	N_a
Total		P_p	N_p	n

- **Negative Predictive Value (NPV)** is the proportion of correctly classified negative examples among all predictions made by the model as negative

$$\text{NPV} = \frac{TN}{N_p} = \frac{TN}{TN + FN}$$

		Predicted values		Total
		Positive	Negative	
Actual values	Positive	TP	FN	P_a
	Negative	FP	TN	N_a
Total		P_p	N_p	n

Model evaluation

Performance metrics for classification

- **Recall**, also known as **Sensitivity** or **True Positive Rate (TPR)**, is the proportion of correctly classified positive examples out of the total number of actual positive examples in the test set

$$\text{Recall} = \text{TPR} = \frac{TP}{P_a} = \frac{TP}{TP + FN}$$

		Predicted values		Total
		Positive	Negative	
Actual values	Positive	TP	FN	P_a
	Negative	FP	TN	N_a
Total		P_p	N_p	n

- **Specificity**, also known as **Selectivity** or **True Negative Rate (TNR)**, is the proportion of correctly classified negative examples out of the total number of actual negative examples in the set

$$\text{Specificity} = \text{TNR} = \frac{TN}{N_a} = \frac{TN}{TN + FP}$$

		Predicted values		Total
		Positive	Negative	
Actual values	Positive	TP	FN	P_a
	Negative	FP	TN	N_a
Total		P_p	N_p	n

Model evaluation

Performance metrics for classification

- **False Discovery Rate (FDR)** is the proportion of incorrectly classified positive examples among all the predicted positive examples

$$FDR = \frac{FP}{P_p} = \frac{FP}{TP + FP}$$

		Predicted values		Total
		Positive	Negative	
Actual values	Positive	TP	FN	P_a
	Negative	FP	TN	N_a
Total		P_p	N_p	n

- **False Omission Rate (FOR)** is the proportion of incorrectly classified negative examples among all the predicted negative examples

$$FOR = \frac{FN}{N_p} = \frac{FN}{FN + TN}$$

		Predicted values		Total
		Positive	Negative	
Actual values	Positive	TP	FN	P_a
	Negative	FP	TN	N_a
Total		P_p	N_p	n

Model evaluation

Performance metrics for classification

- **False Positive Rate** (FPR) is the proportion of actual negative examples that are incorrectly classified as positive

$$FPR = \frac{FP}{N_a} = \frac{FP}{FP + TN}$$

		Predicted values		Total
		Positive	Negative	
Actual values	Positive	TP	FN	P_a
	Negative	FP	TN	N_a
Total		P_p	N_p	n

- **False Negative Rate** (FNR) is the proportion of actual positive examples that are incorrectly classified as negative

$$FNR = \frac{FN}{P_a} = \frac{FN}{FN + TP}$$

		Predicted values		Total
		Positive	Negative	
Actual values	Positive	TP	FN	P_a
	Negative	FP	TN	N_a
Total		P_p	N_p	n

Model evaluation

Performance metrics for classification

- Among all confusion-matrix metrics, **Precision** and **Recall** are typically the most frequently presented
- They **focus on the positive class**, which usually represents the event of interest (e.g., fraud, disease, attack, dropout), often rare and operationally critical
- They explicitly separate the **two error types**:
 - Precision → controls False Positives
 - Recall → controls False Negatives
- They remain meaningful under class imbalance, where metrics like Accuracy can be misleading

Model evaluation

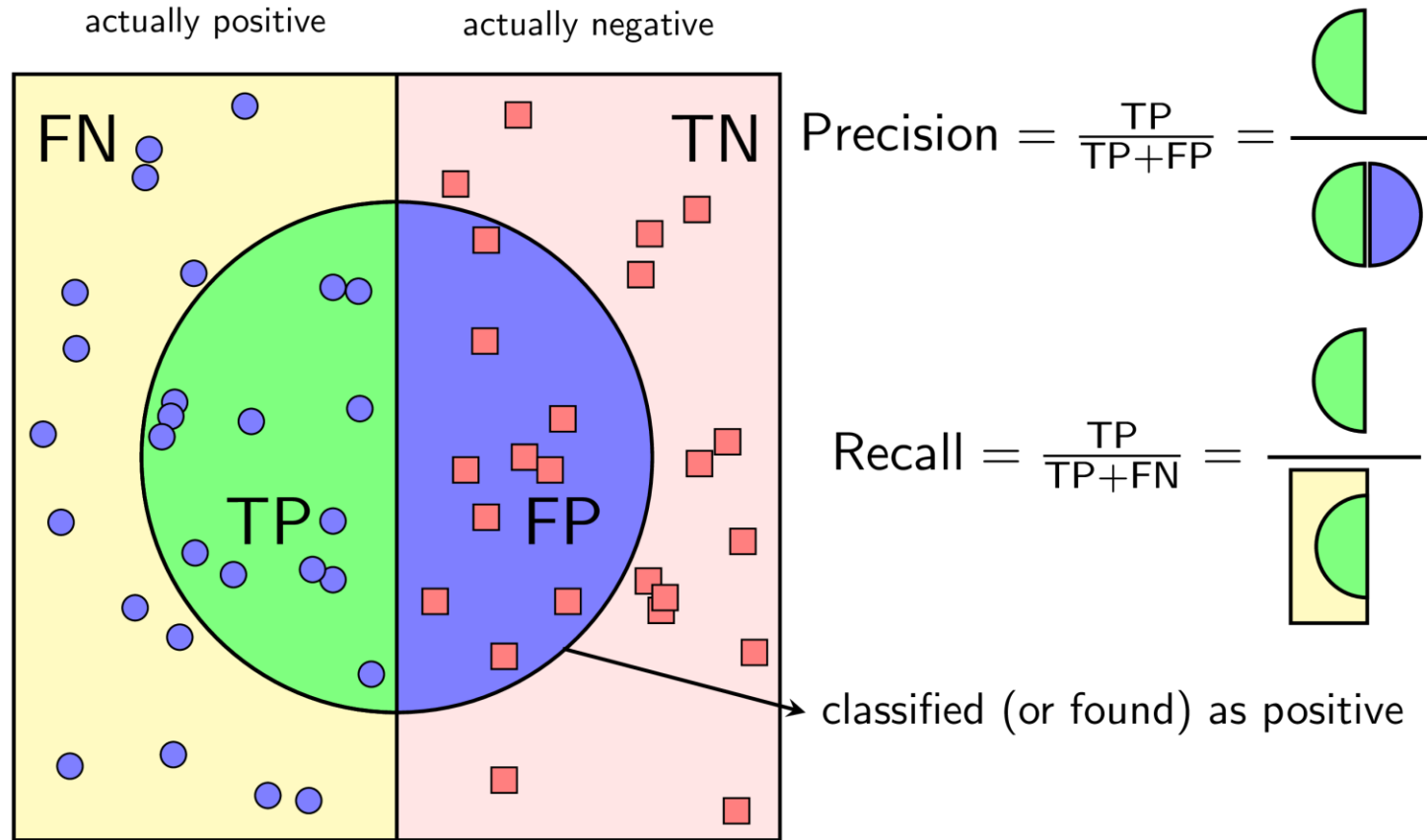
Performance metrics for classification

- They reflect practical **decision trade-offs**:
 - High Recall → fewer missed critical events
 - High Precision → fewer false alarms
- They are **intuitive and easy** to communicate:
 - “Of what we predicted as positive, how much was correct?”
 - “Of what was actually positive, how much did we detect?”
- Therefore, they **offer greater interpretability as evaluation metrics**

Model evaluation

Performance metrics for classification

- A visual depiction



Model evaluation

Performance metrics for classification

- There is an inherent *trade-off* between **Precision** and **Recall**, restraining the simultaneous optimization of both metrics
 - They often exhibit an **inverse relationship**, implying that improving one metric typically comes at the expense of the other
- Recognizing the **limitations** of evaluating precision and recall in isolation, alternative approaches have been proposed
 - One strategy involves comparing values for one metric at a **fixed level of the other**
 - Additionally, efforts have been directed toward combining precision and recall into a single, **unified metric for optimization**

Model evaluation

Performance metrics for classification

- Emerging within the field of information retrieval, the **F_1 score** exemplifies the strategy to unify the two metrics by calculating the harmonic mean of **Precision** and **Recall**

$$F_1 = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{2TP}{2TP + FP + FN}$$

		Predicted values		Total
		Positive	Negative	
Actual values	Positive	TP	FN	P_a
	Negative	FP	TN	N_a
Total		P_p	N_p	n

Model evaluation

Performance metrics for classification

- The F_1 score is a particular case of the parametric family of *F-measures*, denoted by F_β

$$F_\beta = \frac{(1 + \beta^2) \text{Precision} \times \text{Recall}}{\beta^2 \text{Precision} + \text{Recall}}$$

- This family of metrics assigns a weighting factor of β to **Recall** relative to **Precision**, allowing for a flexible assessment of classification performance based on the relative importance of these two measures
- Typical instances of the F_β measure include:
 - F_2 , where **Recall** is given twice the weight of Precision
 - $F_{0.5}$, where **Precision** is allocated twice the weight of Recall

Model evaluation

Performance metrics for classification

- **Cohen's Kappa (κ)** is another metric for assessing agreement between two models, frequently used in binary classification tasks

$$\kappa = \frac{P_o - P_e}{1 - P_e} = \frac{2(TP \times TN - FP \times FN)}{(TP + FP)(FP + TN) + (TP + FN)(FN + TN)}$$

		Predicted values		Total
		Positive	Negative	
Actual values	Positive	TP	FN	P_a
	Negative	FP	TN	N_a
Total		P_p	N_p	n

Model evaluation

Performance metrics for classification

- **Cohen's Kappa (κ):**

$$\kappa = \frac{P_0 - P_e}{1 - P_e} = \frac{2(TP \times TN - FP \times FN)}{(TP + FP)(FP + TN) + (TP + FN)(FN + TN)}$$

- Where:

Observed agreement	$P_0 = \frac{TP + TN}{TP + TN + FP + FN} = Accuracy$
---------------------------	--

Expected agreement	$P_e = \frac{(TP + FP)(TP + FN) + (FP + TN)(FN + TN)}{(TP + TN + FP + FN)^2}$
---------------------------	---

- It conceptualizes models as **raters**: one providing the *actual values* and the other offering the *predictions*

Model evaluation

Performance metrics for classification

- κ is recognized as a chance-corrected alternative to **Accuracy**, addressing the potential for agreement occurring by chance alone
- Within the formula, the observed agreement (P_0) aligns with **Accuracy** and represents the proportion of correct predictions
- The expected agreement (P_e) represents the **agreement expected by chance**, considering the distribution of actual positives and negatives in the test set
- Ranges from -1 to 1 , where:
 - 1 signifies perfect agreement
 - 0 indicates no agreement equal to chance
 - -1 represents performance worse than chance

Model evaluation

Performance metrics for classification

- The **Matthews Correlation Coefficient** (MCC) evaluates the correlation between predicted and actual binary classifications, encompassing all four possible outcomes

$$MCC = \frac{TP \times TN - FP \times FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}}$$

		Predicted values		Total
		Positive	Negative	
Actual values	Positive	TP	FN	P_a
	Negative	FP	TN	N_a
Total		P_p	N_p	n

Model evaluation

Performance metrics for classification

- **Matthews Correlation Coefficient (MCC):**

$$MCC = \frac{TP \times TN - FP \times FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}}$$

- Ranges from -1 to 1 , where:
 - 1 signifies total agreement
 - 0 indicates no better than random
 - -1 represents total disagreement
- Unlike the F_1 score, MCC exhibits class symmetry (interchanging positive and negative classes yields the same MCC value)

Model evaluation

Performance metrics for classification

- Sometimes MCC is preferred as performance metric, compared to:
 - $Accuracy \rightarrow$ ignores class imbalance
 - $F_1 \rightarrow$ ignores true negatives
 - $\kappa \rightarrow$ adjusts for chance agreement
- MCC uses all four confusion matrix terms symmetrically
- Remains informative under strong class imbalance
- Has a statistical interpretation as correlation

Model evaluation

Case studies

- 1st case – Balanced dataset

		Predicted values		Total
		Positive	Negative	
Actual values	Positive	40	10	50
	Negative	10	40	50
Total		50	50	100

- Metric calculations:

- $$Accuracy = \frac{TP+TN}{TP+TN+FP+FN} = \frac{80}{100} = 0.80$$

- $$Precision = \frac{TP}{TP+FP} = \frac{40}{50} = 0.80$$

- $$Recall = \frac{TP}{TP+FN} = \frac{40}{50} = 0.80$$

- $$F_1 = \frac{2 \times Precision \times Recall}{Precision + Recall} = 0.80$$

- $$MCC = \frac{TP \times TN - FP \times FN}{\sqrt{(TP+FP)(TP+FN)(TN+FP)(TN+FN)}} \approx 0.60$$

- Metrics align

- In a balanced dataset the metrics provide consistent evaluation

Model evaluation

Case studies

- **2nd case – Imbalanced dataset**

		Predicted values		Total
		Positive	Negative	
Actual values	Positive	3	2	5
	Negative	4	91	95
Total		7	93	100

- Metric calculations:
 - $Accuracy = 0.94$
 - $Precision = 0.429$
 - $Recall = 0.6$
 - $F_1 = 0.5$
 - $MCC = 0.48$
- $Accuracy$ is misleadingly high due the class imbalance
- F_1 gives a much more realistic view than $Accuracy$
- MCC indicates moderate overall predictive power

Model evaluation

Case studies

- **3rd case – Skewed predictions**

		Predicted values		Total
		Positive	Negative	
Actual values	Positive	8	2	10
	Negative	30	60	90
Total		38	62	100

- Metric calculations:

- $Accuracy = 0.68$
- $Precision = 0.21$
- $Recall = 0.8$
- $F_1 = 0.33$
- $MCC = 0.3$

- $Accuracy$ partially reflects performance but doesn't capture the extreme imbalance in predictions
- F_1 is pulled down because $Precision$ is very low and $Recall$ is high
- MCC provides a more global assessment of model performance, especially when negatives matter

Model evaluation

Case studies

- **4th case – Systematic misclassification**

		Predicted values		Total
		Positive	Negative	
Actual values	Positive	0	10	10
	Negative	10	0	10
Total		10	10	20

- Metric calculations:

- $Accuracy = 0$
- $Precision = 0$
- $Recall = 0$
- $F_1 = 0$
- $MCC = -1$

- $Accuracy$ clearly shows a complete lack of correct predictions
- F_1 confirms that performance on the positive class is completely absent
- MCC reveals when a model is perfectly wrong

Model evaluation

Case studies

- Side-by-side evaluation results

